Lab 10: Software Metrics & Complexity Analysis

Online Doctor Appointment System

A. Lab Details

Project title: Online Doctor Appointment System

Team / roll numbers:

S.No.	Team Member	Roll Number	Responsibility
1	Harsh Mishra	202401100100110	Backend & Database
2	Devansh	202401100100093	Frontend Development
3	Kushagra	202401100100143	Project Planning & Coordination
4	Madhav Agarwal	202401100100146	Testing & Development

Lab objective: To measure the size, complexity and maintainability of the codebase using standard software metrics, and to use those numbers to point out which modules carry the most risk and should be prioritised for testing or refactoring.

***Note:** the figures below follow the project's planned module structure, but they are worked examples to show the method. Before submission, replace them with the real values measured on your own code using a tool such as ESLint's complexity rule, lizard, or SonarQube (Section B lists tools per metric).*

B. Metrics Selected

Category	Metric(s)	What it tells us	Tool / method
Size	Lines of Code (LOC), number of modules/functions	Shows how large each module is, and where effort is concentrated	cloc, sloc
Complexity	Cyclomatic Complexity (CC)	Counts independent paths through a function - flags functions that are hard to test fully	ESLint complexity rule, lizard
Volume / Effort	Halstead Volume, Difficulty, Effort	Estimates how much a function's vocabulary and length make it hard to understand	escomplex / plato (npm), lizard
Maintainability	Maintainability Index (MI)	Combines LOC, CC and Halstead Volume into one 0-100 score of how easy a module is to maintain	Visual Studio Code Metrics, escomplex, formula in Section E
Coupling / Cohesion	Fan-in / Fan-out, module coupling and cohesion level	Flags modules that depend on too much, or are depended on by too much	madge (dependency graph), manual review

C. Size Metrics (Lines of Code)

File	Module	LOC (approx.)	No. of functions
frontend/js/auth.js	Frontend - login & registration forms, session handling	~85	4
frontend/js/doctors.js	Frontend - doctor search, doctor details, slot display	~140	6
frontend/js/booking.js	Frontend - slot selection, booking request	~110	5
frontend/js/dashboard.js	Doctor dashboard - appointment list, status updates	~120	5
backend/routes/appointments.js	Route definitions for appointment endpoints	~50	5
backend/controllers/appointmentController.js	Booking and status business logic	~160	8
backend/controllers/authController.js	Registration and login logic	~95	4
backend/models/Appointment.js	DAO - SQL for the appointment table	~105	6
backend/models/Slot.js	DAO - SQL for the time_slot table	~85	5
backend/middleware/auth.js	Token and role checks	~45	3
backend/server.js	App bootstrap, middleware wiring	~60	4
database/schema.sql	Table definitions	~80	-

Total project size: ~1,135 LOC across 12 core files (excluding generated assets, tests and third-party packages), with 55 functions in total and an average of about 19 LOC per function. The backend (routes, controllers, models, middleware and server) accounts for roughly 53% of total LOC and the front end for about 40%, consistent with the backend holding all the validation and booking logic (login checks, slot availability, double-booking prevention) rather than just data pass-through.

D. Cyclomatic Complexity

Cyclomatic Complexity (CC) = E - N + 2P (edges, nodes, connected components), or equivalently 1 + number of decision points (if/else, &&, ||, for, while, case, catch) in a function. CC of 1-10 is low risk, 11-20 moderate, 21-50 high risk, and above 50 is considered untestable.

Function	File	CC	Comment
validateBookingForm()	frontend/js/booking.js	4	Low - 3 decisions: doctor selected, slot selected, reason not empty
bookAppointment()	frontend/js/booking.js	6	Low - 5 decisions: form invalid, missing token, HTTP error, slot-taken (409) response, network error
AppointmentController.create()	backend/controllers/appointmentController.js	9	Low-moderate - 8 decisions: patient session, missing slot/reason (||), slot exists, slot available, slot not in the past, database error, insert result
AppointmentController.updateStatus()	backend/controllers/appointmentController.js	7	Low - 6 decisions: doctor role, valid status, appointment exists, allowed transition, release slot on cancel, database error
AuthController.login()	backend/controllers/authController.js	6	Low - 5 decisions: missing fields (||), user not found, wrong password, database error

Function	File	CC	Comment
Slot.checkSlot()	backend/models/Slot.js	3	Low - 2 decisions: slot not found, slot already booked
authMiddleware()	backend/middleware/auth.js	4	Low - 3 decisions: token present, token valid, role allowed

All measured functions stay in the low-risk band (CC <= 10). AppointmentController.create() has the highest complexity (CC = 9), as expected - it is the one function that enforces most of the business rules around a valid booking, so it is the first candidate for unit tests and for splitting into smaller validation helpers if it grows further.

E. Halstead Metrics and Maintainability Index

Halstead Volume V = N x log2(n), Difficulty D = (n1 / 2) x (N2 / n2) and Effort E = D x V, where n1/n2 are the distinct operators/operands and N1/N2 the total operators/operands. The Maintainability Index uses the normalised formula MI = max(0, (171 - 5.2 ln V - 0.23 CC - 16.2 ln LOC) x 100 / 171), with function-level LOC and CC.

Function	LOC	CC	Halstead Volume	Halstead Difficulty	Halstead Effort	Maintainability Index (0-100)
AppointmentController.create()	42	9	520	24.0	12,480	44.4
AppointmentController.updateStatus()	30	7	360	18.0	6,480	48.9
AuthController.login()	28	6	300	15.0	4,500	50.3
validateBookingForm()	16	4	150	11.0	1,650	58.0
Slot.checkSlot()	12	3	90	8.0	720	62.4

Using the Visual Studio bands, an MI of 20-100 is good, 10-19 is moderate and 0-9 is poor maintainability. All five functions fall in the good band; AppointmentController.create() has the lowest MI (44.4) and the largest Halstead Effort (12,480), so it is again the function most worth refactoring or covering with tests first.

F. Coupling & Cohesion Summary

Fan-in / fan-out per module (how many modules call it / how many it calls), carried over from the module design in the Software Design Analysis Report:

Module	Fan-in	Fan-out	Comment
User Registration & Login (M1)	3 (M2, M4, M6)	1 (M7)	Highest fan-in together with M7 - every other module needs an authenticated user, so it must stay small and stable
Doctor Search (M2)	1 (M3)	1 (M1)	Simple link in the chain - easy to test with a mocked M1
Slot Management (M3)	2 (M4, M5)	1 (M2)	Shared by booking and cancellation, so check_slot() and release_slot() must stay consistent
Appointment Booking (M4)	1 (M5)	3 (M1, M3, M7)	Highest fan-out - it coordinates login, slot and storage, which matches its high CC
Status Management (M5)	0	2 (M4, M3)	Entry point from the doctor dashboard - expected low fan-in
Admin Data Management (M6)	0	2 (M1, M7)	Entry point from the admin panel, mirrors M5's position

Module	Fan-in	Fan-out	Comment
History & Data / Database (M7)	3 (M1, M4, M6)	0	Leaf module - only ever called, never calls out

G. Conclusion & Recommendations

The metrics agree with each other: AppointmentController.create() in the Appointment Booking module (M4) has the highest cyclomatic complexity (9), the largest Halstead effort and the lowest Maintainability Index, and M4 also has the highest fan-out of any module (3), because it coordinates login, slot checking and storage. It is therefore the single highest-priority target for unit testing and for being split into smaller, single-purpose functions such as validateBookingRequest(), checkSlotAvailability() and saveAppointment() as the project grows.

Recommended actions: (1) write unit tests for every decision point in create() - missing patient, missing reason, unavailable slot, past slot and database failure; (2) add a concurrency test in which two patients book the same slot at the same time, to confirm that double booking is prevented; (3) keep M1 and M7 small and stable, since they have the highest fan-in. This gives a direct, metric-driven confirmation of the module boundaries chosen in the earlier design and architecture labs.